

ATLAS AGENTS ARCHITECTURE

How the AI agent team supporting ATLAS development will work

6

Macs: one per team, plus one for orchestration

5

team Leads, one AI agent per colleague

1

shared farm using the free capacity of all 6 Macs

Status: proposal, version 4. Nothing has been set up yet. Every colleague works with their own AI Lead, and the Leads share a compute farm through a common dashboard. Section 11 adds an alternative scenario. It explains the architecture so the Product team can give feedback before implementation starts.

01 WHAT AND WHY

We want a team of AI agents that works like a small engineering company. Each colleague gets an AI **Lead** for their discipline (Avionics, Structural, Aerodynamics, Mechanical, Product). There is no boss: **every colleague gives instructions only to their own Lead**. The Leads check a shared dashboard to see what is available, and a Chief Engineer agent orchestrates their requests across the spare power of all six Macs, one shared **farm**, so any team can scale up when it needs to.

The team will take on four kinds of work:

- **Designing new versions:** proposes and tests changes on the computer before anything is built.
- **Checking and catching mistakes:** reviews everything before it reaches the real aircraft.
- **Day-to-day work:** settings, simulator tests, reports, housekeeping.
- **Flight preparation:** checklists and flight controller settings before every test.

The agents have **near-full autonomy**: they decide and act on their own, and report afterwards. There are, however, clear red lines (section 07).

02 THREE LAYERS

LAYER	WHO	ROLE
1 Leads	Each colleague and their own AI Lead, on that team's Mac	Colleagues give instructions only to their own Lead. The Lead owns its discipline: plans the work, decides how much help it needs, requests it, merges the results
2 Orchestration	Chief Engineer agent and the dashboard (Mac 1)	Shows what is available, allocates farm capacity to the Leads' requests, coordinates work that spans several teams, settles farm conflicts. It takes requests; it does not give orders
3 Farm	The free capacity of all 6 Macs	Runs the Engineers, Reviewers and simulations that the Leads request, wherever there is room

The Leads

A Lead works on its own: it plans, requests farm resources and runs the work. Its colleague **approves the results** before they leave the team. A Lead never leaves its home Mac.

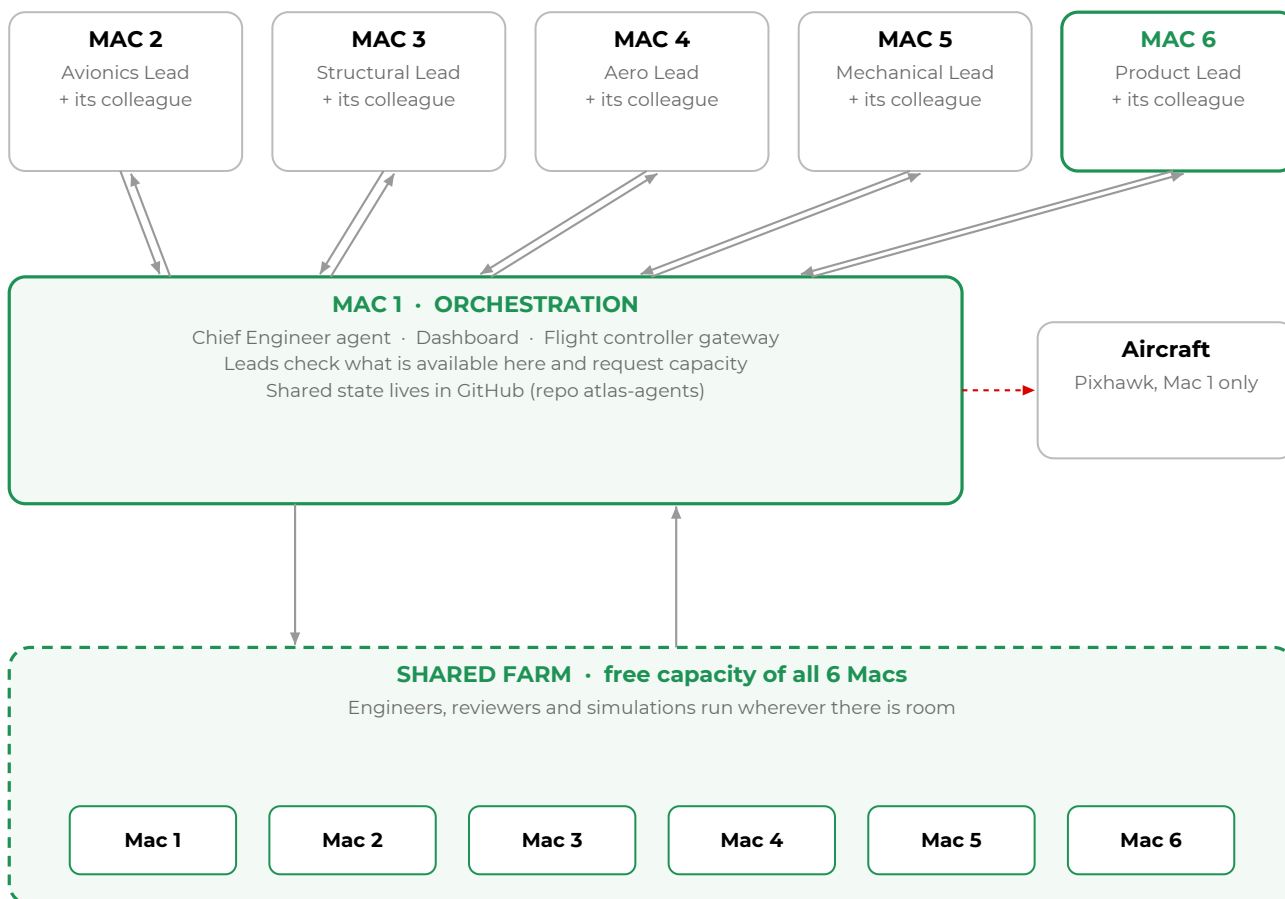
LEAD	MAC	OWNS
Avionics	Mac 2	Flight software, control tuning, sensors and vibration, flight log analysis. Only Lead allowed to send work to the flight controller (via Mac 1)
Structural	Mac 3	Weights and centre of gravity, strength (Fusion 360 Simulation), frame vibration, materials
Aerodynamics	Mac 4	Jetfoils and fan flow, forward-flight stability, airfoil selection, CFD
Mechanical	Mac 5	Fans and motors, batteries and power, assembly and parts, legs and landing
Product	Mac 6	Requirements and goals, schedule and priorities, cost and parts (BOM), reports for external audiences

Engineers and Reviewers

When a Lead needs hands, it starts **Engineers** who do the work and **Reviewers** who check it independently, on the farm. A Reviewer does not fix things itself: it writes down what it found and the work goes back to the Engineer, up to three times, then escalates to the Lead. A Lead can start one pair for a small task, or ten Engineers for a parallel investigation.

03 THE COMPUTERS AND THE FARM

Six Macs in different locations: **one per team, plus Mac 1 for orchestration**. The computers **do not talk to each other directly**. They coordinate only through a shared space on GitHub, so the system works wherever each person is.



Every colleague talks only to their own Lead. Green: orchestration and the farm. Red dashed line: the only connection to the real aircraft, from Mac 1 only.

What the farm actually speeds up

There are two kinds of work, and only one gets much faster with more Macs:

KIND	EXAMPLES	FASTER WITH MORE MACS?
Thinking	AI agents analysing, planning, reviewing	A little. The thinking happens on Anthropic's servers, not on the Mac. The limit here is the Claude account and cost, not hardware
Computing	Flight simulations (SITL), settings sweeps, many flight logs, Blender, CFD preparation	A lot. If one Mac runs 4 simulations at once, 4 Macs run 16

So the biggest win is in simulations. A typical Avionics analysis mixes both: a few agents investigating different causes, plus hundreds of simulated flights spread across the farm.

04 SHARING THE FARM

- **Home Mac guaranteed:** each Lead always keeps its own Mac. The farm uses only spare capacity.
- **Colleague first:** when a colleague needs their Mac, farm work on it pauses and moves elsewhere. Nothing is lost, it just slows down.
- **Priorities:** flight preparation, then urgent, then normal, then background. Higher priority work can take Macs from lower priority work, which pauses and resumes later.
- **Bookings:** a Lead can reserve capacity in advance, e.g. 4 Macs tonight from 20:00 to 08:00.
- **Conflicts:** when two Leads want the same Macs, the **Chief Engineer decides** by the agreed rules. If it cannot, it asks the colleagues involved. Every decision is visible on the dashboard.
- **Same toolbox everywhere:** every Mac has the same simulator and tools installed, so any job can run on any Mac and results are comparable. Anything that touches the aircraft stays on Mac 1.

Example

01	Aero	The Aero Lead starts a 300-simulation sweep of jetfoil angles. The farm gives it 4 free Macs.
02	Avionics	An hour later the Avionics Lead needs an urgent pre-flight check and asks for 2 Macs.
03	Chief	Both want the same Macs, so the Chief Engineer decides: flight preparation wins.
04	Farm	Aero continues on 2 Macs, more slowly. When Avionics finishes, the Macs return to Aero.

05 THE SHARED SPACE

A new private GitHub repo, proposed name **atlas-agents**, for the agents only. ATLAS code and designs stay in their own repos, and agents change them only on their own branches. Large files (flight logs, CFD results) need a shared storage space outside GitHub.

FOLDER	CONTENTS
CHARTER.md	Rules and red lines. Every agent reads this first
leads/	Instructions for each Lead, and for the Engineers and Reviewers it can start
machines/	Each Mac: owner, home Lead, tools, sharing settings
farm/	The job board and the allocation rules
missions/	Goals, e.g. "stable hover of V3_30"
decisions/	Decisions on disagreements and farm conflicts
reports/	Reports from the Leads and the Chief Engineer
worker/, dashboard/	The small "worker" that runs on each Mac, and the dashboard

06 HOW A TASK FLOWS

01	Ask	A colleague gives instructions to their own Lead, in plain language.
02	Check	The Lead checks the dashboard: which Macs are free and what is already running.
03	Plan	The Lead plans the work. If it needs other teams, it asks their Leads through the Chief Engineer.
04	Request	The Lead requests what it needs, e.g. "3 Engineers and 160 simulations".
05	Allocate	The Chief Engineer allocates free Macs by the sharing rules and settles any conflicts.
06	Work	Engineers and simulations run on the farm, each on a clean copy and its own branch.
07	Review	Reviewers check. Problems go back to the Engineer (up to 3 times).
08	Merge	The Lead merges the results and its colleague approves them.
09	Approve	Merging into the main branch happens only with approval from the owners of the affected areas.

The **dashboard** (on Mac 1) shows each Mac (free, partly shared, busy, offline), what each Lead is doing, how the farm is split, what is waiting for approval, and the reports. The team works **only when given a task**, not continuously in the background.

07 RED LINES

No agent crosses these lines, whatever it is asked by text, a file or another agent:

×	Files	No deleting or overwriting. Only new copies and new versions.
×	Main branch	No pushing to main without human approval. Each team pushes freely only to its own branches.
×	Flight controller	Only the Avionics Lead, only through Mac 1. Writing settings and rebooting only while disarmed, always with a backup of the old settings first and a read-back afterwards.
×	Motors	No agent ever arms the aircraft or spins motors. Ever.
×	Colleague's Mac	Agents work only inside their own folder, never touch personal files, and yield the Mac to its owner.
×	Disagreements	When two teams disagree, the Chief Engineer stops and asks the colleagues involved, presenting both options.

08 FIRST MISSION: STABLE HOVER

The first goal is for ATLAS V3_30 to lift off and hover steadily in the real test. Work starts with the Product Lead, because every other team measures against the numbers Product defines.

LEAD	TASK
Product (first)	Defines "stable" in numbers (e.g. stays within X cm and Y degrees for Z seconds) and the test plan
Structural	Confirms the weights and centre of gravity of the real V3_30. Checks whether the frame shakes from the fans
Mechanical	Thrust margin in hover, battery voltage sag, motor temperatures
Aerodynamics	How much the jetfoils and the ground affect hover, and how much control authority they leave
Avionics	Analysis of recent flights, a tuning sweep on the farm with the new weights, flight controller settings
Chief Engineer	One report: "ready to test or not, and why"

09 WHAT WE NEED FROM PRODUCT

- **Feedback on this architecture** before implementation: is a responsibility missing, is a limit wrong?
- **A definition of "stable hover"** with measurable numbers. It is the first task of the mission.
- **Mac 6:** confirmation that the Product Lead can live there and that spare capacity can join the farm, and when (only when idle, or on a schedule).
- **Sources for cost and parts:** where the bill of materials (BOM) and prices live today.
- **Report recipients:** who outside the team (investors, partners) should receive reports, and in what format.

10 IMPLEMENTATION STEPS

PHASE	WHAT HAPPENS
0 Preparation	Push the current work to GitHub, create atlas-agents, the same toolbox on every Mac
1 Pilot	Chief Engineer and Avionics Lead only. Test the red lines with one small task
2 Simulation farm	Simulation sweeps spread across all free Macs. The biggest win with the least risk
3 All Leads	Structural, Aerodynamics, Mechanical and Product Leads on their Macs. Full dashboard
4 Parallel agents	Leads start many Engineers and Reviewers on the farm. First "stable hover" mission

11 ALTERNATIVE SCENARIO

This section describes an **alternative**. Everything in sections 01 to 10 remains the main proposal. The alternative changes two things: **how work is split** (A) and **where the Orchestrator runs** (B). They can be adopted together or separately.

A. Micro-tasks across the farm

Every piece of work is split into many small, independent tasks that run at the same time across all Macs. The **Lead splits the work**, because it knows the discipline: what can run on its own and what has to wait for something else. The **Orchestrator decides where and when** each task runs.

One trap: "very small" can become "slower". Every task has a fixed overhead (setup, sending files, the agent reading its instructions). If a task lasts seconds, more time goes into coordination than into work. A good task:

- **lasts minutes**, not seconds;
- **is independent**: it does not wait for another task halfway through;
- **is self-contained**: everything it needs travels with it;
- **has a clear "done"** and a small, standard result.

Example: Avionics hover analysis in about 185 tasks

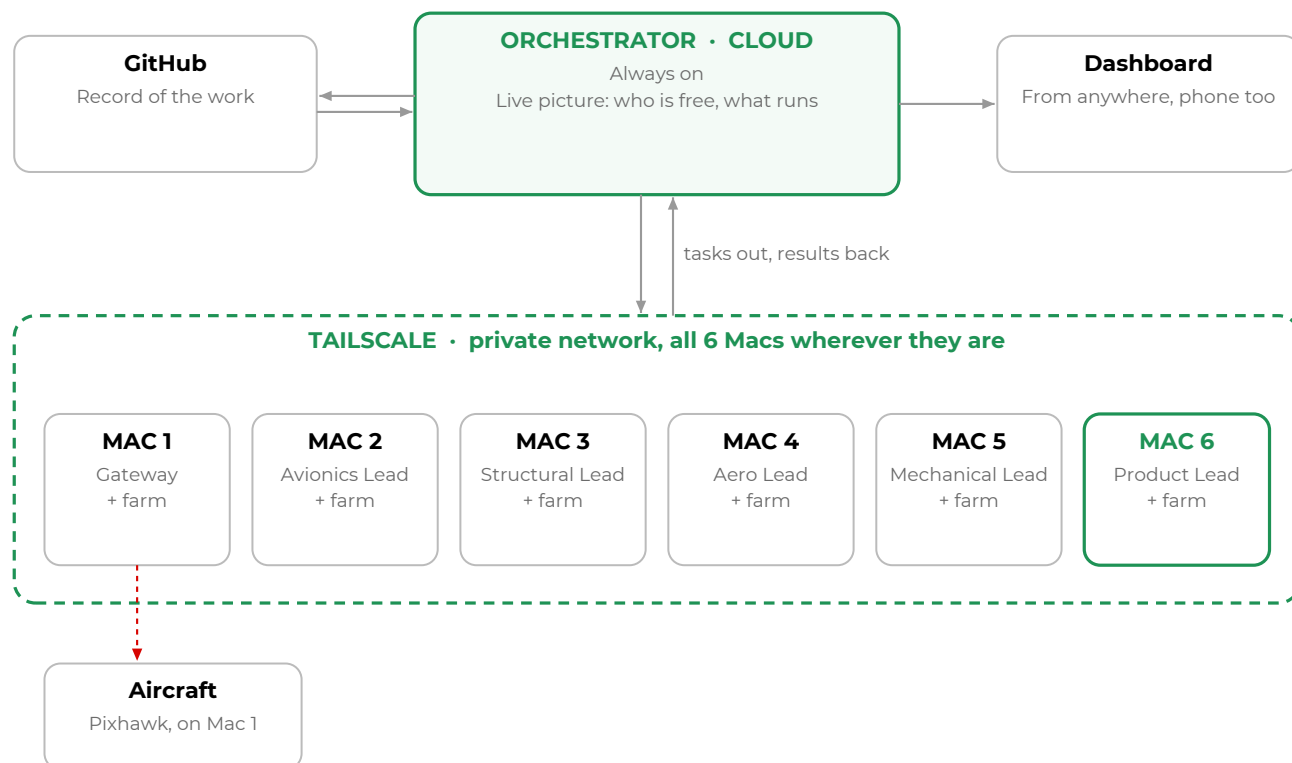
STAGE	TASKS	RUNS
1 Log analysis	12, one per flight	In parallel
2 Hypotheses	4 agents: vibration, centre of gravity, tuning, motors	In parallel
3 Simulations	160 SITL flights, one per settings combination	In parallel on all Macs
4 Review	8 Reviewers	In parallel
5 Synthesis	1 report from the Lead	At the end

Strategic placement by the Orchestrator

- **Right task, right Mac**: Fusion only where there is a licence, the flight controller only on the Mac with the Pixhawk, heavy work on Macs that are plugged in.
- **Keep related tasks together** on the Mac that already has the files, so large logs are not copied back and forth.
- **Long tasks first**, so no single task is left running at the end while everything else waits.
- **Run the last slow tasks twice** on another Mac and keep whichever finishes first.
- **Save progress** inside each task, so it continues elsewhere if a colleague takes their Mac back.

B. Orchestrator in the cloud, connected with Tailscale

The Orchestrator runs on a small cloud server instead of Mac 1. **Tailscale** creates a **private network** between the server and all six Macs, wherever they are, as if they were in the same office.



Green dashed line: the Tailscale private network. Red dashed line: the flight controller stays physically on the Mac it is plugged into.

BENEFIT	WATCH OUT FOR
Always on. MacBooks close their lids and travel; a cloud Orchestrator does not	Security: a cloud machine that can reach every Mac is sensitive. Strict Tailscale rules let it reach only the worker on each Mac, nothing else
Solves mixed locations without a VPN or opening ports	The flight controller does not move to the cloud. Its gateway stays on the Mac the Pixhawk is plugged into
Dashboard from anywhere , including a phone (closes an open question)	If the cloud is down , no new tasks are assigned. Macs finish what they are running, so nothing is lost
Mac 1 is freed to work like every other Mac	A small monthly server cost, and Tailscale installed on every Mac

The split of duties stays clear: the **Orchestrator holds the live picture** (who is free, what is running) and **GitHub holds the record of the work** (code, results, decisions).

12 OPEN QUESTIONS

- When the farm may use a colleague's Mac: only when idle, always with a limit, or on a schedule.
- Claude cost: a budget per team and a decision on accounts.
- Shared storage for large files (flight logs, CFD results).
- The remote server for CFD simulations (to be defined later).
- Whether Fusion 360 Simulation can run automatically, or needs a click from the Structural colleague.
- Whether the dashboard should also be visible outside Mac 1, e.g. from a phone.
- Which scenario to start with: the Orchestrator on Mac 1 (main proposal), or micro-tasks and a cloud Orchestrator with Tailscale (section 11).